SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Nethra P(192511012) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

nethra

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

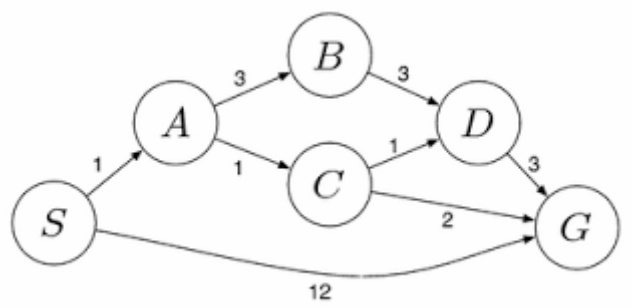
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Q1. The Water Jug Problem

[Marks: 10]

Problem Statement: We are given two unmarked jugs — a 4-gallon jug and a 3-gallon jug — and an unlimited supply of water from a pump. Neither jug has any calibration markings. The objective is to measure out exactly 2 gallons of water using only the following permitted operations:

Permitted Operations

(a) Fill a jug completely from the pump. (b) Empty a jug completely onto the ground. (c) Pour water from one jug into the other until either the source jug is empty or the destination jug is full.

Formulating the Search Problem

Component	Description
State	An ordered pair (x, y), where x = water in the 4-gallon jug (0–4) and y = water in the 3-gallon jug (0–3).
Initial State	(0, 0) — both jugs empty.
Goal State	(2, y) for any value of y — exactly 2 gallons present in the 4-gallon jug.
Operators	Fill 4-gal, Fill 3-gal, Empty 4-gal, Empty 3-gal, Pour 4→3, Pour 3→4.
Path Cost	One unit per operation performed (each fill/empty/pour counts as one step).

Step-by-Step Solution (Shortest Sequence)

Step	Action Performed	4-Gallon Jug	3-Gallon Jug
0	Initial state	0	0
1	Fill the 3-gallon jug from the pump	0	3
2	Pour water from 3-gallon jug into 4-gallon jug	3	0
3	Fill the 3-gallon jug again from the pump	3	3
4	Pour from 3-gallon jug into 4-gallon jug until it is full (only 1 gallon is accepted)	4	2
5	Empty the 4-gallon jug completely on the ground	0	2
6	Pour the remaining water from 3-gallon jug into 4-gallon jug	2	0

After Step 6, the 4-gallon jug contains exactly 2 gallons of water, which satisfies the goal state (2, y). This sequence of six operations is the minimum-length solution obtained by applying breadth-first search over the state space of the problem.

Q2. Mars Rover — Intelligent Agent Analysis

[Marks: 10]

A Mars Rover can be modelled as an intelligent agent whose task is to explore the Martian surface, collect soil/rock samples, and transmit scientific data back to Earth. Its behaviour is analysed below using the PEAS framework and agent-architecture theory.

(i) Percepts of the Agent

The percepts are the raw sensory inputs the rover receives from its environment at any instant, including:

Sensor / Source	Percept Received
Stereo Cameras	Images of terrain, rocks, and obstacles
Spectrometers	Chemical/mineral composition of rocks and soil
GPS / Inertial Unit	Current position, orientation and tilt
Proximity / LIDAR Sensors	Distance to nearby obstacles
Temperature & Radiation Sensors	Atmospheric temperature and radiation levels
Wheel Odometry	Distance travelled and wheel slippage

(ii) Characterizing the Operating Environment

Property	Nature for Mars Rover
Observability	Partially observable — sensors cover only the visible/nearby terrain
Number of Agents	Single-agent (occasionally multi-agent if multiple rovers cooperate)
Determinism	Stochastic — wheel slip, dust storms and terrain are unpredictable
Episodic / Sequential	Sequential — current actions affect future navigation and sampling
Static / Dynamic	Dynamic — weather, lighting and terrain can change during operation
Discrete / Continuous	Continuous — position, speed and sensor readings are continuous
Known / Unknown	Unknown — the exact terrain map of Mars is not fully known in advance

(iii) Actions the Agent Can Take

Move forward / backward, turn left / right, stop; capture images; drill into rock; scoop soil sample; operate onboard spectrometer/analyser; store sample in collection chamber; transmit data to orbiter/Earth; enter low-power/sleep mode; avoid or navigate around an obstacle.

(iv) Evaluating the Agent's Performance

The performance measure quantifies how well the rover achieves its mission goals. Typical performance metrics include:

Performance Criterion	Explanation
Distance safely traversed	Total ground covered without collision or damage
Number & quality of samples analysed	Scientific value of the data collected
Energy efficiency	Battery/solar power consumed per task completed
Data transmission accuracy	Correctness and completeness of data sent to Earth
Mission time	Time taken to complete assigned objectives
Survivability	Ability to avoid hazards such as pits, slopes, and dust storms

(v) Most Suitable Agent Architecture

A Learning, Utility-Based Agent built on a Model-Based Reflex core is the most suitable architecture for the Mars Rover. This is because:

• Model-based: The environment is partially observable, so the rover must maintain an internal model (map) of the terrain built from past percepts to decide the next action sensibly.

• Utility-based: Multiple valid actions may exist (e.g., several safe paths); the rover must choose the action that maximises scientific value while minimising risk and energy cost — this requires a utility function rather than a single fixed goal test.

• Learning: Since Mars terrain is largely unknown and communication delay with Earth (10–20 minutes) prevents real-time human control, the rover must learn from experience — refining its terrain model and navigation policy autonomously over time.

Q3. The 8-Queens Problem — Search Space & Strategies

[Marks: 10]

The 8-Queens problem requires placing 8 queens on a standard 8×8 chessboard such that no two queens attack each other — i.e., no two queens share the same row, column, or diagonal. A common formulation places exactly one queen per column, so only the row of each queen needs to be decided.

Problem Components

Component	Description
Initial State	A configuration with 8 queens already placed, one per column (as given in the figure). This is NOT a solution because the queen in the first column shares a diagonal with the queen in the last column.
States	Any arrangement of 0 to 8 queens on the board, one per column, in any row.
Actions	Move a queen already on the board to a different row within the same column (in the incremental-repair formulation), or add a queen to the left-most empty column (in the constructive formulation).
Transition Model	Given a state and an action, returns the resulting board configuration after the queen's row is changed / a new queen is added.
Goal Test	No two queens attack each other, i.e., no two share a row, column, or diagonal.
Path Cost	Not significant for this problem — every step costs the same (constant cost); we only care about reaching a valid goal state.

Search Space of the Problem

If one queen is restricted to each column, each of the 8 queens can occupy any of 8 possible rows, giving a search space of $8^8 = 16{,}777{,}216$ possible board configurations. If no such column restriction is imposed, the unrestricted search space is C(64, 8), which is drastically larger — hence the column-restriction is a critical piece of problem-formulation that shrinks the search space.

Search Strategies Applicable to 8-Queens

Strategy	How It Applies to 8-Queens
Backtracking (Depth-First Search with pruning)	Places queens column-by-column; whenever a queen attacks a previously placed queen, the algorithm backtracks to the previous column and tries another row.
Hill-Climbing	Starts from a random full configuration (as given in the figure) and repeatedly moves a queen to reduce the number of attacking pairs until no improving move exists.
Min-Conflicts Heuristic	A local-search variant: repeatedly selects a conflicted queen and moves it to the row that minimises the number of attacks — very effective for 8-Queens and scales to large N.

Strategy	How It Applies to 8-Queens
Simulated Annealing	Similar to hill-climbing but occasionally accepts worse moves (controlled by a 'temperature' parameter) to escape local minima.
Genetic Algorithm	Maintains a population of board configurations, combining (crossover) and mutating them across generations, retaining configurations with fewer attacking pairs.

Since the given configuration in the figure already has one queen per column but is not a valid solution (first and last column queens share a diagonal), a local-search approach such as Hill-Climbing or Min-Conflicts is particularly appropriate here — it can repair the existing configuration by moving individual queens rather than restarting the search from scratch.

Q4. OLA Cab Booking — Agent Type & Pseudocode

[Marks: 10]

In the OLA Cab mobile application, a customer wants to travel from one location to another and can choose from several cab categories — Mini, Sedan, Shared, Prime, and so on — based on personal comfort, budget and urgency.

Identifying the Type of Problem-Solving Agent

This scenario best fits a Utility-Based Agent (built upon an underlying Goal-Based / search agent for route-finding). A simple goal-based agent can only tell whether a plan reaches the destination or not — it cannot distinguish between multiple valid options such as Mini, Sedan, or Prime. Since the customer must choose the option that best balances cost, comfort, waiting time and travel time, the agent needs a utility function to rank the available choices and select the one that maximises overall customer satisfaction.

PEAS Element	Description for OLA Cab Agent
Performance Measure	Minimised cost, minimised waiting/travel time, maximised comfort & customer rating
Environment	City road network, live traffic, available drivers, weather
Actuators	Cab booking confirmation, route/navigation instructions to driver, fare display
Sensors	Customer's pickup/drop GPS location, real-time traffic data, driver availability feed

Pseudocode for the OLA Cab Booking Agent

```
FUNCTION OLA_CAB_AGENT(pickup, destination, preferences):
    cab_types <- {Mini, Sedan, Shared, Prime, Auto}
    percept <- SENSE(pickup, destination, live_traffic, driver_locations)
    candidate_list <- EMPTY LIST

    FOR EACH type IN cab_types DO
        IF driver_available(type, pickup) THEN
            fare        <- ESTIMATE_FARE(type, pickup, destination, live_traffic)
            eta         <- ESTIMATE_TIME(type, pickup, driver_locations)
            comfort     <- COMFORT_RATING(type)
            utility     <- W1*(1/fare) + W2*(1/eta) + W3*comfort
            ADD (type, fare, eta, utility) TO candidate_list
        END IF
    END FOR

    best_cab <- ARGMAX(candidate_list, BY utility)
    ROUTE <- UCS_OR_A*_SEARCH(pickup, destination)   // least-cost path
    CONFIRM_BOOKING(best_cab, ROUTE)
    RETURN best_cab, ROUTE, fare, eta
END FUNCTION
```

Here W1, W2, W3 are weights reflecting how much the customer values price, speed, and comfort respectively, and the ARGMAX step is precisely what makes this a utility-based agent rather than a plain goal-based one.

Q5. Logistics Delivery Network — Uniform Cost Search (UCS)

[Marks: 10]

The delivery network is represented as a weighted graph with warehouses S, A, B, C, D and G as nodes, and edges labelled with the combined fuel + time cost of travelling that route. The goal is to find the least-cost path from the starting warehouse S to the destination warehouse G using Uniform Cost Search (UCS).

Edges Extracted from the Network Diagram

Edge	Cost
S – A	1
S – G (direct route)	12
A – B	3
A – C	1
B – D	3
C – D	1
C – G	2
D – G	3

How Uniform Cost Search Works

UCS expands the node with the lowest cumulative path cost g(n) from the start node first (using a priority queue / min-heap frontier), rather than expanding nodes purely by depth or breadth. It continues until the goal node is popped from the frontier, guaranteeing the least-cost path in a graph with non-negative edge weights.

Step-by-Step UCS Trace (Frontier Expansion)

Step	Node Expanded	Frontier After Expansion (node : cumulative cost)
1	S (cost 0)	A : 1, G : 12
2	A (cost 1)	C : 2, B : 4, G : 12
3	C (cost 2)	D : 3, B : 4, G : 4 (updated via S-A-C-G, replacing 12)
4	D (cost 3)	B : 4, G : 4 (S-A-C-D-G costs 6, so G stays at 4)
5	G (cost 4) — Goal reached!	Search terminates

Result

Candidate Path	Total Cost
S → A → C → G	1 + 1 + 2 = 4 ✓ (Least-Cost / Optimal Path)
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → G (direct)	12

Optimal Path: S → A → C → G Minimum Total Cost = 4 units

Hence, the logistics company should route packages from warehouse S to G via A and C, achieving the lowest combined fuel-and-time cost of 4 units, instead of the direct S–G route which costs 12 units.

— End of Annexure A —